

JARVIS

MVP Feature Spec

Spécifications complètes de toutes les fonctionnalités du MVP — critères de done, scope exact, exclusions

Version : 1.0 · Dossier : 02_MVP / MFS-001 · Mai 2026

Ce document est la source de vérité unique pour le développement Phase 1. Toute fonctionnalité non listée ici est hors scope MVP. Toute demande d'ajout doit passer par une révision formelle de ce document avant d'être implémentée. Le scope est figé.

1. Résumé du scope MVP

Catégorie	Fonctionnalités	Priorité globale
Interface HUD	Design Iron Man animé, chat, voix STT, dashboard	Critique
Authentification	JWT + OAuth Google (Gmail + Calendar)	Critique
Chat IA	Conversation avec Claude API, streaming SSE, contexte	Critique
Mémoire	3 niveaux : Redis + pgvector + PostgreSQL, RAG pipeline	Critique
Gestion tâches	CRUD, extraction depuis chat, priorisation, dashboard	Critique
Projets	Contextes isolés, switch depuis l'interface	Haute
Morning Briefing	Synthèse quotidienne : emails, agenda, tâches, météo	Haute
Contrôle PC	Volume, apps, mode boulot, captures, stats système	Haute
Musique	Spotify + YouTube via API	Haute
Voix STT	Web Speech API, transcription navigateur, commandes	Haute
Google Intégrations	Gmail lecture + Calendar lecture/écriture	Haute
Recherche web	SerpAPI ou Tavily, résultats injectés dans le contexte	Haute
Sécurité	Chiffrement mémoire, RLS PostgreSQL, rate limiting	Critique
Monitoring	Logs LLM, tokens, coûts, latence	Haute

2. Spécifications détaillées par fonctionnalité

2.1 Interface utilisateur — HUD JARVIS

Design Iron Man animé · Next.js · TailwindCSS

F-001 — Interface HUD animée style Iron Man

Priorité : Critique · **Phase :** Phase 1d

Description : Interface web immersive avec cercles concentriques animés (canvas 2D), effet de scan rotatif, points pulsants sur l'anneau extérieur, centre avec nom J.A.R.V.I.S et status. Fond #020d1a, couleurs cyan #00d4ff. Design validé en prototype.

Critères de done :

- Canvas HUD animé tourne en continu sans latence perceptible
- Animations fluides à 60fps sur un PC standard
- Texte J.A.R.V.I.S centré avec effet de glow pulsant
- Status dynamique (STANDBY / ACTIF / ÉCOUTE) mis à jour en temps réel
- Interface responsive — fonctionne de 1024px à 1920px

Exclu du scope : Animations Three.js ou WebGL — canvas 2D suffit pour le MVP

F-002 — Chat intégré au HUD

Priorité : Critique · **Phase :** Phase 1b

Description : Zone de chat sous le HUD avec historique des messages, input texte, bouton envoi, streaming des réponses token par token via SSE. Messages colorés : utilisateur en cyan foncé, JARVIS en cyan vif.

Critères de done :

- Messages s'affichent avec streaming visible token par token
- Historique scrollable des 30 derniers messages
- Input avec focus automatique au chargement
- Touche Entrée envoie le message
- Indicateur visuel pendant que JARVIS génère sa réponse

Exclu du scope : Historique de plus de 30 messages dans l'interface — le reste est en mémoire backend

F-003 — Dashboard système — stats PC

Priorité : Haute · **Phase :** Phase 1d

Description : 4 métriques temps réel sous le HUD : CPU%, RAM%, Heure, Température système. Barres de progression colorées (vert → orange → rouge selon valeur). Mise à jour toutes les 2 secondes via endpoint backend.

Critères de done :

- CPU et RAM récupérés depuis le backend (psutil Python)
- Heure locale affichée en HH:MM
- Température système si disponible (psutil sensors)
- Barres changent de couleur selon les seuils définis
- Mise à jour automatique sans rechargement de page

Exclu du scope : Graphiques historiques — simple valeur instantanée suffit pour le MVP

F-004 — Panneau de contrôle PC — 12 boutons

Priorité : Haute · **Phase :** Phase 1d

Description : 3 colonnes de 4 boutons chacune : Système PC (volume+, volume-, capture, gestionnaire), Applications (Chrome, Spotify, VS Code, mode boulot), IA JARVIS (Morning Briefing, Focus, Recherche web, Mes tâches). Chaque bouton appelle un endpoint backend qui exécute l'action.

Critères de done :

- Chaque bouton déclenche une action réelle via appel API backend
- Retour visuel immédiat (bouton s'illumine + message dans le chat)
- Commande volume via pyautogui ou nircmd (Windows)
- Lancement d'applications via subprocess Python
- Mode boulot : lance Chrome + Spotify + VS Code en une commande

Exclu du scope : Contrôle de toutes les applications installées — seulement les 4 prédéfinies en MVP

2.2 Authentification & Sécurité

JWT · OAuth2 · Google · Multi-utilisateurs

F-005 — Authentification JWT

Priorité : Critique · **Phase :** Phase 1a

Description : Login email/password avec hashage bcrypt, émission JWT (access token 1h + refresh token 7j stocké en httpOnly cookie), validation automatique à chaque requête via middleware FastAPI.

Critères de done :

- Login retourne access_token JWT valide 1h
- Refresh token httpOnly cookie renouvelle silencieusement l'access token
- Logout invalide le refresh token côté serveur
- Toutes les routes protégées retournent 401 si JWT absent ou expiré
- Multi-utilisateurs : isolation stricte par user_id sur toutes les données

Exclu du scope : SSO entreprise, SAML, magic links — JWT classique suffit pour le MVP

F-006 — OAuth Google — Gmail + Calendar

Priorité : Haute · **Phase :** Phase 1d

Description : Flow OAuth 2.0 Google complet pour obtenir accès à Gmail (lecture) et Calendar (lecture + écriture). Tokens stockés chiffrés en base. Refresh automatique avant expiration.

Critères de done :

- Bouton 'Connecter Google' dans les paramètres initie le flow OAuth
- Consentement Google affiche les scopes Gmail readonly + Calendar
- Callback enregistre access_token et refresh_token chiffrés en base
- JARVIS peut lire les 10 derniers emails non lus
- JARVIS peut lire l'agenda du jour et créer des événements
- Refresh automatique si access_token expiré

Exclu du scope : Modification d'emails, envoi d'emails, Google Drive — V2

2.3 Intelligence IA — LLM & Orchestration

Claude API · LLM Router · LangChain

F-007 — LLM Router — Routage intelligent

Priorité : Critique

Phase : Phase 1b

Description : Composant central qui choisit le bon modèle selon le type de requête : Claude Sonnet pour le raisonnement complexe, Gemini 2.5 Flash pour les tâches légères, Groq pour la rapidité, Ollama pour le mode offline.

Critères de done :

- Chaque requête est classifiée avant d'être routée (classify_task function)
- Règles de routing documentées et testées pour chaque type de tâche
- Failover automatique : Claude → Gemini → Groq → Ollama
- Chaque appel LLM loggé avec modèle, tokens, latence, coût estimé
- Test de disponibilité de chaque provider au démarrage du système

Exclu du scope : Interface d'administration du routeur, tuning des règles via UI — V2

F-008 — Personnalité JARVIS — System prompt

Priorité : Haute

Phase : Phase 1b

Description : JARVIS a une personnalité définie : efficace, légèrement sarcastique, direct, jamais condescendant. Personnalité configurable par l'utilisateur (formel / normal / sarcastique) depuis les paramètres.

Critères de done :

- System prompt définit clairement la personnalité et les règles de comportement
- 3 modes de personnalité sélectionnables : Formel, Normal, Sarcastique
- Le mode sélectionné persiste dans le profil utilisateur
- JARVIS utilise le prénom de l'utilisateur s'il est connu
- Les réponses respectent la personnalité choisie de façon cohérente

Exclu du scope : Création de personnalités custom, voix différentes par personnalité — V2

F-009 — Streaming SSE des réponses

Priorité : Critique

Phase : Phase 1b

Description : Les réponses LLM sont streamées token par token via Server-Sent Events depuis le backend FastAPI vers le frontend Next.js. L'utilisateur voit la réponse s'afficher en temps réel sans attendre la réponse complète.

Critères de done :

- Premier token affiché en moins de 1500ms
- Streaming visible et fluide sans saccades
- Arrêt propre du stream si l'utilisateur navigue ailleurs
- Gestion des erreurs : message d'erreur si le stream est interrompu
- Indicateur de génération (animation) pendant le streaming

Exclu du scope : Interruption manuelle du stream par l'utilisateur — V2

2.4 Mémoire — 3 niveaux

Redis · pgvector · PostgreSQL · RAG

F-010 — Mémoire court terme — Redis

Priorité : Critique

Phase : Phase 1c

Description : Les 30 derniers messages de chaque session utilisateur sont stockés dans Redis

avec TTL de 2h. Rechargés automatiquement depuis PostgreSQL si Redis expire.

Critères de done :

- 30 derniers messages stockés et récupérés en < 5ms
- TTL 2h réinitialisé à chaque nouveau message
- Si Redis expire : rechargement automatique depuis la dernière conversation PostgreSQL
- Messages structurés : role, content, timestamp, tokens
- LTRIM automatique pour maintenir la limite des 30 messages

F-011 — Mémoire épisodique — pgvector + RAG

Priorité : Critique · **Phase :** Phase 1c

Description : Les faits importants sur l'utilisateur sont vectorisés (embedding 1536D) et stockés dans PostgreSQL avec pgvector. À chaque requête, les 5 faits les plus pertinents sont récupérés par recherche sémantique.

Critères de done :

- Embedding généré pour chaque nouveau fait (text-embedding-3-small ou Ollama nomic-embed)
- Recherche sémantique retourne les top-5 faits en < 50ms via index HNSW
- Faits injectés dans le system prompt avant chaque appel LLM
- Extraction automatique de nouveaux faits après chaque réponse JARVIS
- Déduplication : similarité > 0.92 → mise à jour plutôt que doublon
- Limite de 500 faits par utilisateur avec nettoyage des moins importants

Exclu du scope : Mémoire collective entre utilisateurs, partage de contexte — V3

F-012 — Mémoire de travail — PostgreSQL

Priorité : Critique · **Phase :** Phase 1c

Description : L'état opérationnel du jour : tâches actives, agenda, projets en cours. Injecté dans chaque requête et dans le Morning Briefing.

Critères de done :

- Tâches actives et prioritaires du jour récupérées à chaque requête
- Projets actifs avec leur résumé récupérés si pertinents
- Morning Briefing sauvegardé en base et récupéré si déjà généré aujourd'hui
- Contexte mis à jour immédiatement quand une tâche est créée ou complétée
- Injection limitée à 800 tokens pour préserver le budget

F-013 — Commandes mémoire utilisateur

Priorité : Haute · **Phase :** Phase 1c

Description : L'utilisateur peut interagir directement avec sa mémoire via des commandes naturelles en chat ou via l'interface dédiée.

Critères de done :

- 'Mémorise que...' → crée un nouveau fait en mémoire épisodique immédiatement
- 'Oublie...' → supprime le fait correspondant après confirmation
- 'Qu'est-ce que tu sais sur moi ?' → liste les 20 faits les plus importants
- 'Efface toute ma mémoire' → suppression complète avec double confirmation
- Interface dédiée dans les paramètres pour voir et gérer tous les faits

2.5 Gestion des tâches

CRUD · Extraction automatique · Priorisation

F-014 — CRUD complet des tâches

Priorité : Critique · **Phase :** Phase 1d

Description : Création, lecture, modification et suppression des tâches depuis le chat ou depuis le dashboard. Les tâches sont associées à un projet, une priorité et une date d'échéance optionnelle.

Critères de done :

- Créer une tâche depuis le chat : 'Ajoute une tâche : finir la doc JARVIS'
- Lister les tâches : 'Montre mes tâches du jour' ou depuis le dashboard
- Marquer comme terminé depuis le chat ou depuis l'interface
- Modifier priorité et date d'échéance
- Supprimer une tâche avec confirmation
- Filtres : par projet, par priorité, par statut, par date

Exclu du scope : Sous-tâches, dépendances entre tâches, Kanban board — V2

F-015 — Extraction automatique de tâches

Priorité : Haute · **Phase :** Phase 1d

Description : JARVIS détecte automatiquement quand une conversation implique une tâche à faire et propose de l'ajouter à la liste sans que l'utilisateur le demande explicitement.

Critères de done :

- Détection dans chaque réponse des formulations impliquant une action à faire
- Proposition explicite : 'Voulez-vous que j'ajoute ça à vos tâches ?'
- L'utilisateur répond oui/non — pas d'ajout automatique sans confirmation
- La tâche est pré-remplie avec le bon titre et la bonne priorité estimée
- Lié au projet actif si un projet est sélectionné

Exclu du scope : Extraction automatique sans confirmation — trop intrusif pour le MVP

2.6 Morning Briefing

Agrégation multi-sources · Synthèse IA · Personnalisé

F-016 — Morning Briefing automatique

Priorité : Haute · **Phase :** Phase 1d

Description : Synthèse personnalisée générée au premier accès de la journée. Agrège : emails Gmail non lus, agenda du jour, tâches prioritaires, météo, état des projets actifs.

Critères de done :

- Généré automatiquement au premier accès de chaque journée
- Récupère les 10 derniers emails Gmail non lus et les résume
- Affiche les événements Calendar du jour et du lendemain
- Liste les 3 tâches les plus prioritaires du jour
- Météo du lieu configuré dans le profil utilisateur (API wttr.in)
- Résumé des projets actifs avec dernière mise à jour
- Sauvegardé en base pour ne pas être régénéré si l'utilisateur rafraîchit
- Accessible manuellement via le bouton 'Morning Briefing' du panneau

Exclu du scope : Briefing audio vocal, envoi par email ou notification push — V2

2.7 Contrôle système PC

Windows · pyautogui · psutil · subprocess

F-017 — Contrôle volume et système

Priorité : Haute · **Phase :** Phase 1d

Description : JARVIS peut contrôler le volume système, capturer l'écran, ouvrir le gestionnaire des tâches et fournir des informations sur l'état du PC via des commandes en chat ou via les boutons du panneau.

Critères de done :

- Volume + / - / mute via pyautogui ou nircmd (Windows) / osascript (Mac)
- Capture d'écran sauvegardée dans le dossier téléchargements avec timestamp
- Informations système : CPU%, RAM%, uptime, température via psutil
- Ouverture gestionnaire des tâches Windows (Ctrl+Shift+Esc via pyautogui)
- Heure et date disponibles sans appel LLM (réponse directe < 10ms)

Exclu du scope : Contrôle total de toutes les applications et processus — V2

F-018 — Lancement d'applications

Priorité : Haute · **Phase :** Phase 1d

Description : JARVIS peut ouvrir les applications prédéfinies sur commande vocale ou textuelle, et activer des modes de travail qui lancent plusieurs applications en une commande.

Critères de done :

- Chrome, Spotify, VS Code, Notepad lancés via subprocess Python
- Mode boulot : lance les 3 apps définies par l'utilisateur en une commande
- Mode boulot configurable depuis les paramètres (liste des apps à lancer)
- Confirmation dans le chat que l'application est lancée
- Détection si l'application est déjà ouverte (pas de double lancement)

Exclu du scope : Fermeture d'applications, contrôle de toutes les apps installées — V2

2.8 Musique — Spotify & YouTube

Spotify Web API · YouTube Data API v3

F-019 — Contrôle Spotify

Priorité : Haute · **Phase :** Phase 1d

Description : JARVIS contrôle Spotify via la Spotify Web API : lecture, pause, titre suivant, précédent, volume, recherche par artiste ou titre.

Critères de done :

- Lecture / pause / suivant / précédent via commandes chat ou boutons
- 'Mets du [artiste]' → recherche et lance l'artiste sur Spotify
- 'Monte le volume Spotify à 70%' → contrôle du volume indépendant du système
- Affichage du titre en cours dans l'interface JARVIS
- Auth Spotify OAuth 2.0 (même flow que Google)

Exclu du scope : Gestion des playlists, mode shuffle, repeat — V2

F-020 — Lancement YouTube

Priorité : Haute · **Phase :** Phase 1d

Description : JARVIS peut lancer une vidéo YouTube par titre ou artiste en ouvrant l'URL dans le navigateur par défaut.

Critères de done :

- 'Lance [titre] sur YouTube' → recherche via YouTube Data API v3
- URL de la première vidéo pertinente ouverte dans le navigateur
- Confirmation dans le chat avec le titre de la vidéo trouvée
- Gestion des erreurs si la vidéo n'est pas trouvée
- Clé API YouTube Data v3 configurée dans le .env

Exclu du scope : Lecteur YouTube intégré dans l'interface JARVIS — V2

2.9 Voix — Web Speech API

STT navigateur · Transcription FR · Commandes vocales

F-021 — Reconnaissance vocale STT

Priorité : Critique · **Phase :** Phase 1d

Description : Input vocal via Web Speech API natif du navigateur. L'utilisateur clique sur le bouton microphone, parle, la transcription est envoyée au backend comme un message texte classique.

Critères de done :

- Bouton microphone dans l'interface — clic pour activer/désactiver
- Transcription en français (lang='fr-FR') en temps réel visible dans l'input
- Message envoyé automatiquement après détection de fin de parole
- Feedback visuel : barres animées pendant l'écoute active
- Fallback élégant si Web Speech API non supporté : message d'instruction
- Commandes vocales reconnues : toutes les fonctionnalités accessibles par texte

Exclu du scope : TTS (synthèse vocale des réponses), wake word, mode Iron Man — V2

2.10 Projets et Contextes

Isolation de mémoire · Switch de contexte

F-022 — Gestion des projets

Priorité : Haute · **Phase :** Phase 1d

Description : Chaque projet a sa propre mémoire isolée. L'utilisateur peut switcher de contexte de projet et JARVIS adapte son contexte en conséquence.

Critères de done :

- Création de projets avec nom et description
- Switch de projet depuis un sélecteur dans l'interface HUD
- La mémoire épisodique filtrée par projet_id lors de la recherche vectorielle
- Résumé du projet injecté dans le contexte quand le projet est actif
- Liste de tous les projets accessible depuis les paramètres
- Archivage d'un projet (masqué mais conservé)

Exclu du scope : Partage de projet entre utilisateurs, permissions, collaboration — V3

2.11 Recherche web

SerpAPI / Tavily · Injection dans le contexte

F-023 — Recherche web temps réel

Priorité : Haute · **Phase** : Phase 1d

Description : JARVIS peut effectuer une recherche web et injecter les résultats dans le contexte de sa réponse pour répondre à des questions nécessitant des informations actuelles.

Critères de done :

- Détection automatique des requêtes nécessitant une recherche web
- Appel SerpAPI ou Tavily avec la requête reformulée par JARVIS
- Top 3 résultats injectés dans le contexte (titre + extrait)
- JARVIS cite ses sources dans la réponse
- Fallback : si pas de résultats, répond sur ses connaissances en précisant la date de coupure
- Coût SerpAPI loggé et imputable à l'utilisateur

Exclu du scope : Navigation web complète, extraction de contenu de pages — V2

3. Exclusions explicites du MVP

Les fonctionnalités suivantes sont explicitement exclues du MVP. Leur exclusion est délibérée et documentée. Aucune ne doit être implémentée avant livraison complète de la Phase 1.

Fonctionnalité exclue	Raison de l'exclusion	Phase prévue
TTS — Synthèse vocale des réponses	Coût ElevenLabs/OpenAI TTS + latence non justifiés avant validation du concept	V2
Mode Iron Man — always listening	Stabilité microphone insuffisante, daemon local complexe	V2
Application mobile native	Effort frontend disproportionné pour le MVP	V2
Home Assistant — domotique	Non installé dans le setup actuel — connecteur préparé architecturalement	V2
Intégrations Slack / Notion / GitHub	Une seule intégration externe validée suffit pour le MVP (Google)	V2
Orchestration multi-agents	Architecturalement prématurée avant stabilisation du système mémoire	V3
Proactivité système (actions sans sollicitation)	Nécessite un daemon background et un système d'événements — V3	V3
Vision caméra / analyse d'écran	Complexité élevée, non prioritaire pour valider le concept core	V3
Google Drive	Gmail + Calendar suffisent pour valider l'intégration Google	V2
Export de mémoire	Fonctionnalité utile mais non bloquante pour le MVP	V2

4. Ordre de développement Phase 1

L'ordre de développement est imposé par les dépendances entre composants. Chaque sous-phase doit être terminée et testée avant de commencer la suivante.

Sous-phase	Durée	Fonctionnalités à livrer	Dépendances
1a — Fondations	2 sem.	Docker Compose, PostgreSQL + pgvector, Redis, FastAPI skeleton, F-005 Auth JWT	Aucune — point de départ
1b — LLM Core	2 sem.	F-007 LLM Router, F-008 Personnalité, F-009 Streaming SSE, Chat basique connecté	Phase 1a terminée
1c — Mémoire	2 sem.	F-010 Redis N1, F-011 pgvector N2, F-012 Travail N3, F-013 Commandes mémoire	Phase 1b terminée + pgvector configuré
1d — Features	2 sem.	F-001 à F-004 HUD, F-006 OAuth Google, F-014 à F-023 toutes les features MVP	Phase 1c terminée
1e — Polish	1 sem.	Tests end-to-end, correction bugs, optimisation performances, doc utilisateur	Phase 1d terminée

5. Critères de done — Phase 1 complète

La Phase 1 (MVP) est considérée comme terminée quand TOUS les critères suivants sont satisfaits simultanément.

- Le stack complet démarre en une commande (docker-compose up) sans erreur
- Un utilisateur peut créer un compte, se connecter et accéder à l'interface HUD
- JARVIS répond à une question en moins de 3 secondes (streaming visible < 1.5s)
- JARVIS se souvient d'un fait d'une session précédente sans que l'utilisateur le répète
- Le Morning Briefing affiche emails + agenda + tâches + météo du jour
- Les 12 boutons du panneau PC déclenchent les actions correspondantes
- La voix fonctionne : parler → JARVIS répond
- Spotify se contrôle depuis le chat ou les boutons
- Gmail et Calendar sont connectés et lisibles depuis JARVIS
- Les tâches peuvent être créées, modifiées et complétées depuis le chat
- Les projets peuvent être créés et switchés depuis l'interface
- Aucune fuite de données entre utilisateurs (tests d'isolation)
- Les tokens LLM consommés sont loggés et consultables
- Le code est dans un repository GitHub avec README d'installation